

SE RTU Solution 2024

Part A

Q1. Define software. Enlist the characteristics of good software.

Software is a collection of computer programs, procedures, documentation, and related data that enables a computer system to perform specific tasks.

Characteristics of good software:

- **Correctness** – Performs required functions accurately
- **Reliability** – Operates without failure
- **Efficiency** – Uses minimum resources
- **Maintainability** – Easy to modify and update
- **Usability** – Easy to understand and use
- **Portability** – Can run on different platforms

Q2. Give the difference between Function Point (FP) and Lines of Code (LOC).

Aspect	FP	LOC
Measurement	Functionality-based	Size-based
Dependency	Language independent	Language dependent
Stage	Can be estimated early	Known after coding
Accuracy	More reliable	Less reliable
Used for	Effort estimation	Productivity measurement

Q3. What is SRS?

SRS (Software Requirement Specification) is a formal document that describes:

- Functional requirements
- Non-functional requirements
- System constraints
- Interfaces and performance needs

It acts as a **contract between customer and developer** and serves as a base for design, coding, and testing.

SE RTU Solution 2024

Q4. Explain FSM model.

The **Finite State Machine (FSM) model** represents system behavior using:

1. States
2. Transitions
3. Inputs
4. Outputs

A system moves from one state to another based on input conditions. FSM is widely used in **control systems, protocols, and real-time applications**.

Q5. Why is accuracy an important attribute for data dictionaries?

Accuracy in a data dictionary ensures:

- Correct definition of data elements
- Consistency across the system
- Avoidance of ambiguity and redundancy
- Reliable database design and system implementation

Inaccurate data dictionaries lead to **design errors and faulty software behavior**.

Q6. What is software design? Write any four design principles.

Software design is the process of transforming requirements into a structured solution that defines architecture, components, interfaces, and data flow.

Design principles:

1. **Modularity** – Divide system into modules
2. **Abstraction** – Focus on essential features
3. **Low Coupling** – Minimum interdependence
4. **High Cohesion** – Related functions grouped together

Q7. What is Input/Process/Output (IPO) approach in software design?

The **IPO approach** describes a system in three steps:

- 1) **Input** – Data entered into the system
- 2) **Process** – Logic or computation applied

SE RTU Solution 2024

3) **Output** – Result produced

This approach helps in **structured design and clarity of system functionality**.

Q8. What do you mean by Object-Oriented (OO) concept? Write three OO principles.

The **Object-Oriented concept** models software as a collection of objects that represent real-world entities.

OO principles:

1. **Encapsulation** – Binding data and methods together
2. **Inheritance** – Reusing existing classes
3. **Polymorphism** – One interface, multiple implementations

Q9. Explain Risk Analysis. Enlist four major categories of risk analysis.

Risk analysis is the process of identifying, evaluating, and managing potential problems that may affect project success.

Major risk categories:

- a. **Technical risks** – Technology or design issues
- b. **Schedule risks** – Project delays
- c. **Cost risks** – Budget overruns
- d. **Operational risks** – Deployment and maintenance issues

Q10. Differentiate between Object-Oriented Analysis (OOA) and Object-Oriented Design (OOD).

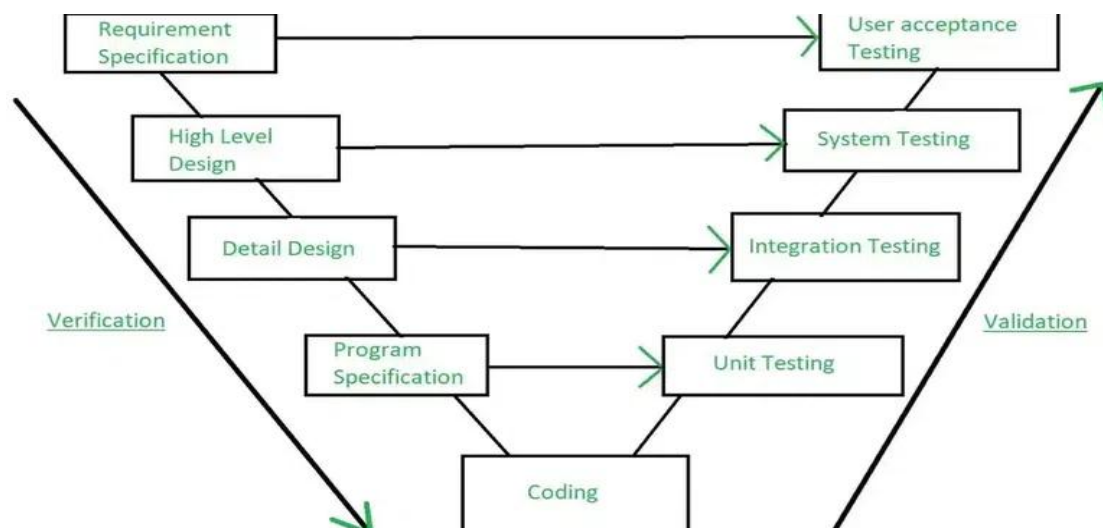
OOA	OOD
Focuses on problem analysis	Focuses on solution design
Identifies objects and requirements	Defines classes and architecture
What system should do	How system will be built
User-oriented	Developer-oriented

SE RTU Solution 2024

Part B

Q1. Difference between Verification and Validation with diagram and example

Properties	Verification	Validation
Meaning	Process of evaluating work products to check whether they meet specified requirements	Process of evaluating the final product to check whether it satisfies user needs
Main Question Answered	<i>Are we building the product correctly?</i>	<i>Are we building the correct product?</i>
Objective	To ensure correctness, completeness, and consistency of specifications	To ensure customer satisfaction and usability
Nature of Process	Static process	Dynamic process
Code Execution	Does not involve executing code	Involves execution of code
Stage Performed	Early stages of development	Later stages of development
Focus Area	Internal quality (design, logic, documentation)	External quality (functionality, performance, usability)
Based On	Requirement specifications, design documents	User requirements and real-world scenarios
Activities Involved	Reviews, inspections, walkthroughs, desk checking	Testing (unit, integration, system, acceptance testing)
Who Performs It	Developers, designers, QA team	Testers, end users, customers
Output	Verified documents and code	Validated, working product
Example	Reviewing requirement document for missing features	Testing login feature to see if user can log in successfully



SE RTU Solution 2024

Q2. Write a short note on Object-Oriented Design (OOD) concepts

Object-Oriented Design (OOD) is a software design approach that models a system as a collection of interacting **objects**, where each object represents a real-world entity. OOD helps in building **reusable**, **maintainable**, and **scalable** software systems.

Key OOD Concepts

- **Object** : An object is an instance of a class that contains **data (attributes)** and **behavior (methods)**.
- **Class** : A class is a blueprint or template used to create objects.
- **Encapsulation** : It binds data and methods together and hides internal details using access specifiers.
- **Abstraction** : It shows only essential features to the user and hides unnecessary implementation details.
- **Inheritance**: It allows one class to acquire properties and methods of another class, promoting code reuse.
- **Polymorphism** : It allows the same method name to behave differently based on the object or context.

Advantages of OOD

1. Improves code reusability
2. Enhances security and data hiding
3. Makes maintenance and debugging easier
4. Closely represents real-world systems

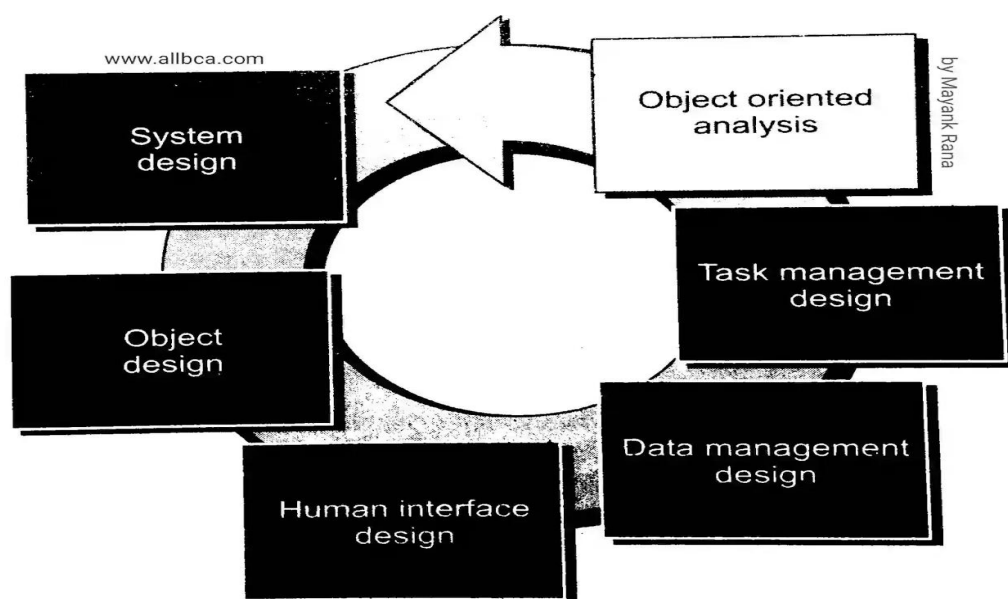


Fig. Process flow for object-oriented design.

SE RTU Solution 2024

Q3. Difference between DFD and CFD with example and diagram

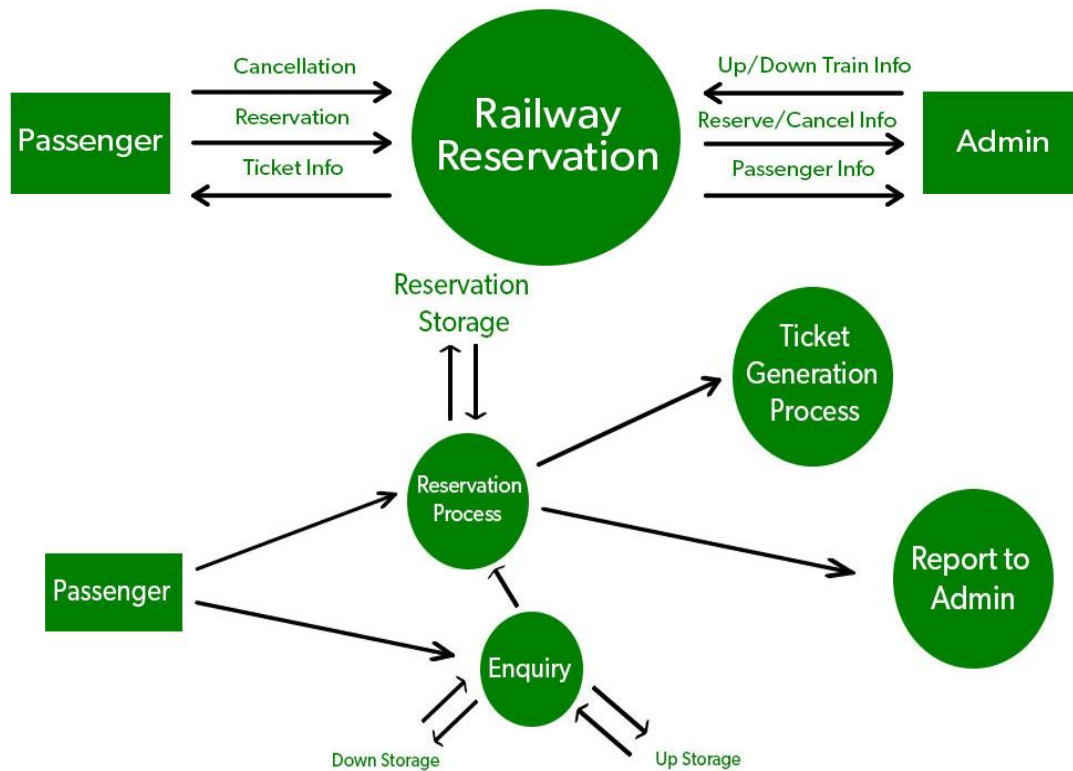
Both **DFD (Data Flow Diagram)** and **CFD (Control Flow Diagram)** are used in **system and software design**, but they focus on **different aspects** of a system. DFD emphasizes **data movement**, while CFD emphasizes **control and decision flow**.

Parameter	DFD (Data Flow Diagram)	CFD (Control Flow Diagram)
Full Form	Data Flow Diagram	Control Flow Diagram
Main Focus	Flow of data	Flow of control and decisions
Purpose	Shows how data is processed and stored	Shows order of execution and decision-making
Represents	Input, processing, output, and storage of data	Conditions, loops, and sequence of operations
Emphasis On	<i>What data flows through the system</i>	<i>When and how control flows</i>
Symbols Used	Process, Data flow, Data store, External entity	Decision boxes, control arrows, conditions
Decision Making	Not shown explicitly	Clearly shown using conditions
Timing & Sequence	Does not show timing	Shows execution sequence
Leveling	Supports leveling (Level 0, Level 1, etc.)	Usually no leveling
Used In	Requirement analysis and system design	Program logic and control analysis
Example Use Case	Banking system, library system	Login validation, algorithm steps

Example of DFD (Ticket Booking System)

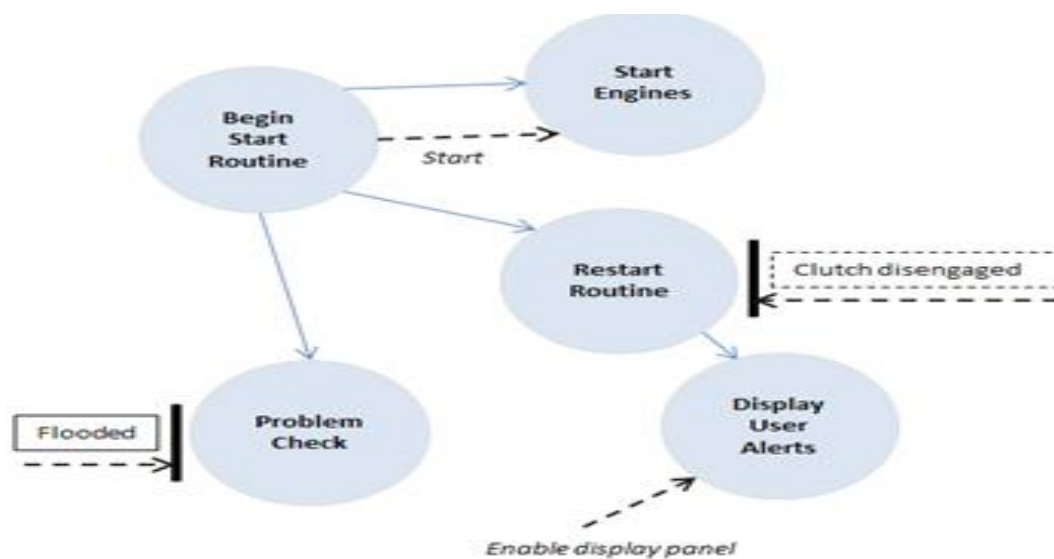
- Passenger enters journey details
- System checks seat availability
- Passenger enters personal details Payment is processed
- Ticket is confirmed and issued

SE RTU Solution 2024



Example of CFD (Engine System)

- Begin start routine
- Start engine
- Check for problem
- If flooded → restart routine
- Disengage clutch
- Display user alert



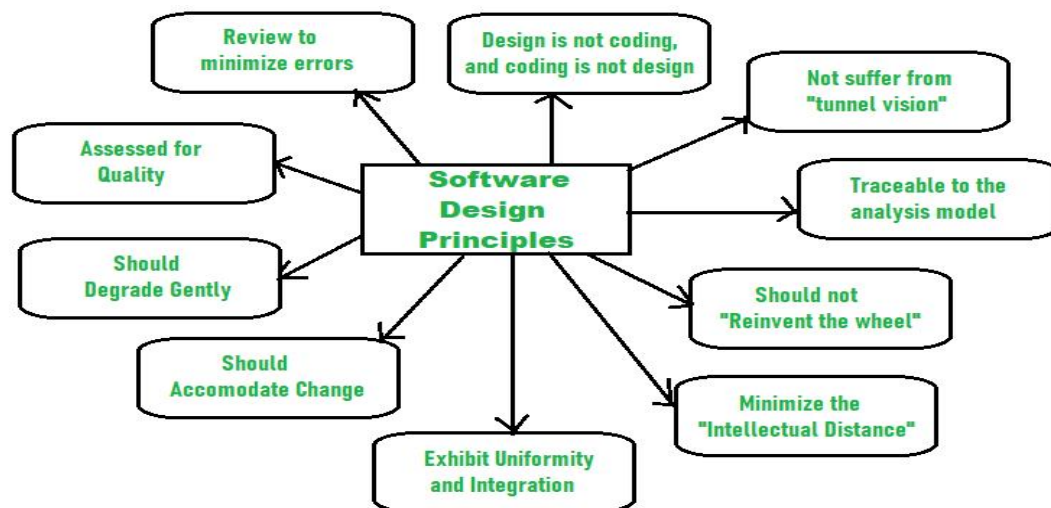
SE RTU Solution 2024

Q4. What is good software design? Explain design documentation with example

Good software design is the process of creating a well-structured solution that satisfies **user requirements**, is **easy to understand**, **maintain**, **modify**, and **scale**. It acts as a **blueprint** for developers before coding starts.

Characteristics of Good Software Design

- **Correctness** – Meets all functional requirements
- **Simplicity** – Easy to understand and implement
- **Modularity** – Divided into independent modules
- **Reusability** – Components can be reused
- **Maintainability** – Easy to update and fix errors
- **Efficiency** – Optimal use of time and resources
- **Scalability** – Can handle future growth



Design Documentation : Design documentation is a written description of how the software system will be built. It explains the architecture, modules, data flow, and control flow of the system. It helps developers, testers, and stakeholders understand the system clearly.

Contents of Design Documentation

- System architecture
- Module description
- Data structures
- DFD and CFD
- Interface design
- Database design

SE RTU Solution 2024

Q5. Explain Software Development Life Cycle (SDLC) with diagram

The **Software Development Life Cycle (SDLC)** is a **systematic process** used to design, develop, test, deploy, and maintain software applications. It provides a **structured approach** that helps in delivering **high-quality, reliable, and cost-effective software**.

Phases of SDLC

1. Requirement Analysis

- Collects requirements from users and stakeholders

- Defines functional and non-functional requirements

- Output: **Software Requirement Specification (SRS)**

2. System Design

- Converts requirements into system architecture

- Includes database design, DFD, CFD, and interface design

- Output: **Design Document**

3. Implementation (Coding)

- Actual coding of the software using programming languages

- Each module is developed separately

- Output: **Source Code**

4. Testing

- Checks software for errors and defects

- Includes unit testing, integration testing, system testing

- Ensures software meets requirements

5. Deployment

- Software is released to users

- Can be full deployment or phased deployment

SE RTU Solution 2024

6. Maintenance

Bug fixing and performance improvement

Enhancements based on user feedback

Longest phase of SDLC

SDLC Diagram



Advantages of SDLC

- Improves software quality
- Reduces development risk
- Better project management
- Clear documentation

SE RTU Solution 2024

Q6. What is prototyping? Explain sequence of events

Prototyping is a software development approach in which a **working model (prototype)** of the software is built to understand and clarify **user requirements**. The prototype is an **early version of the system** that demonstrates functionality, user interface, and system flow before final development.

Prototyping is especially useful when requirements are **not clearly defined** or may change frequently. It helps reduce misunderstandings between users and developers.

Sequence of Events in Prototyping Model

a) Requirement Gathering

- Initial requirements are collected from users
- Requirements may be incomplete or unclear

b) Quick Design

- A simple design is created focusing on **user interface and main features**
- No detailed internal logic is designed at this stage

c) Build Prototype

- A working prototype is developed quickly
- Shows screens, inputs, outputs, and basic functionality

d) User Evaluation

- Users interact with the prototype
- Feedback is collected regarding functionality, design, and usability

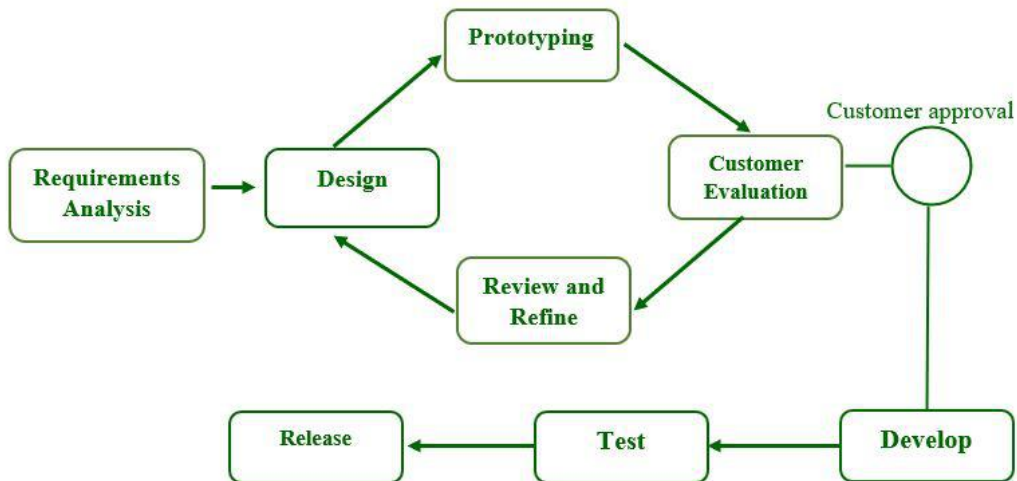
e) Refinement of Prototype

- Prototype is improved based on user feedback
- Steps 3 and 4 may repeat multiple times

f) Final System Development

- Once users approve the prototype, the **actual system** is developed
- Proper coding, testing, and documentation are done

SE RTU Solution 2024



Advantages of Prototyping

- Better requirement understanding
- Early user involvement
- Reduced development errors
- Improved user satisfaction

Q7. Suppose a project was estimated to be 400 KLOC.

Mode	a	b	c	d
Organic	2.4	1.05	2.5	0.38
Semi-Detached	3.0	1.12	2.5	0.35
Embedded	3.6	1.20	2.5	0.32

Calculate Effort and Development Time for each of the three modes of development.

This question is based on the **Basic COCOMO Model**.

Basic COCOMO Formulas

Effort (E) in Person-Months

$$E = a \times (KLOC)^b$$

Development Time (T) in Months

$$T = c \times E^d$$

SE RTU Solution 2024

Where **a, b, c, d** depend on the development mode.

$$\text{KLOC} = 400$$

1. Organic Mode

Effort

$$E = 2.4 \times (400)^{1.05}$$

$$E \approx 2.4 \times 539.9 = 1296 \text{ PM}$$

Development Time

$$T = 2.5 \times (1296)^{0.38}$$

$$T \approx 38.5 \text{ months}$$

2. Semi-Detached Mode

Effort

$$E = 3.0 \times (400)^{1.12}$$

$$E \approx 3.0 \times 824.6 = 2474 \text{ PM}$$

Development Time

$$T = 2.5 \times (2474)^{0.35}$$

$$T \approx 38.4 \text{ months}$$

3. Embedded Mode

Effort

$$E = 3.6 \times (400)^{1.20}$$

$$E \approx 3.6 \times 1326.6 = 4776 \text{ PM}$$

Development Time

$$T = 2.5 \times (4776)^{0.32}$$

$$T \approx 37.7 \text{ months}$$

SE RTU Solution 2024

Part C

Q1. Explain Spiral Model of Software Development with labelled diagram. State advantages and disadvantages.

The **Spiral Model** is a **risk-driven software development model** proposed by Barry Boehm. It combines features of the **Waterfall model** and **Prototyping**, with strong emphasis on **risk analysis**. Development proceeds in a **spiral of loops**, where each loop represents a phase of the software process. At every iteration, risks are identified and resolved before moving forward.

Phases of the Spiral Model (Each Loop)

Each spiral loop is divided into **four main quadrants**:

1. Planning (Objective Setting)

- Define objectives for the current phase
- Identify alternatives and constraints
- Decide goals such as performance, functionality, or cost

2. Risk Analysis

- Identify technical, managerial, and schedule risks
- Evaluate alternatives to reduce risks
- Often involves **prototyping** to clarify requirements

3. Engineering (Development & Testing)

- Actual design, coding, and testing are carried out
- Develop software incrementally
- Verify and validate the developed components

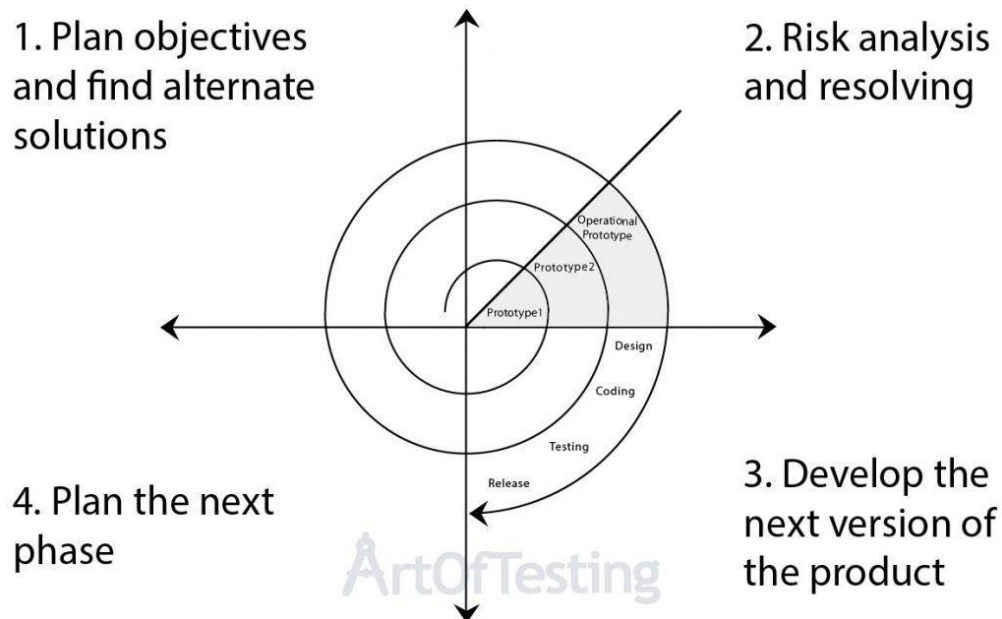
4. Evaluation (Customer Review)

- Customer reviews the developed version
- Feedback is collected
- Decision is made whether to continue to next iteration

This cycle repeats until the final software product is completed.

SE RTU Solution 2024

Spiral Model – Labelled Diagram



Advantages of Spiral Model

- Strong **risk management** at every stage
- Early detection and reduction of errors
- Suitable for **large and complex projects**
- Continuous user feedback improves requirement accuracy
- Flexible to changes during development

Disadvantages of Spiral Model

- Complex to manage and implement
- High cost due to repeated risk analysis
- Requires experienced personnel
- Not suitable for small or low-risk projects
- Difficult to schedule accurately

SE RTU Solution 2024

Q2. What do you mean by DFD? Explain its types with proper diagram. Draw Level-0 and Level-1 DFD for College Registration System.

A **Data Flow Diagram (DFD)** graphically represents the **flow of data** within a system, showing how data moves from input to processing and finally to output.

DFD answers the question:

“Where does data come from, how is it processed, and where does it go?”

Purpose of DFD

- To analyze **data processing**
- To understand **system functionality**
- To identify **data sources and destinations**
- To improve communication with users

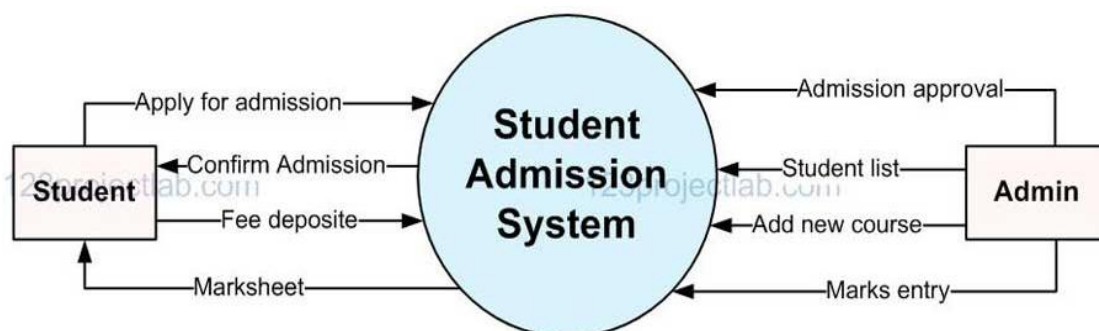
Types of DFD

Level-0 DFD (Context Diagram) : Shows the entire system as a **single process**.

Level-1 DFD : Breaks Level-0 into **sub-processes**.

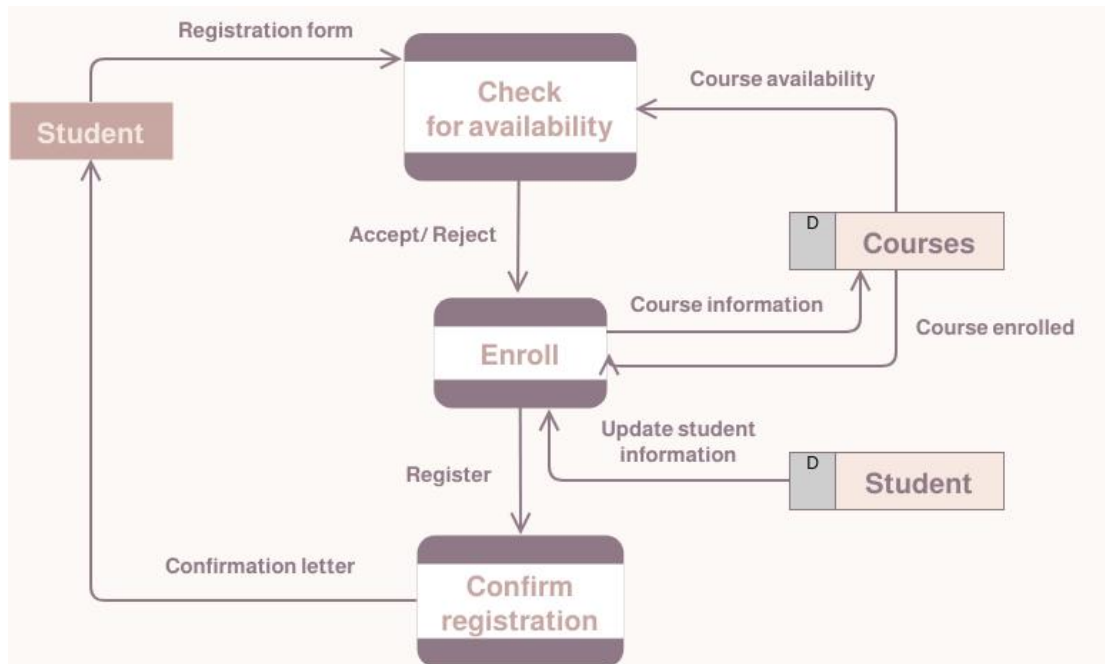
Level-2 DFD (Detailed)

Level-0 DFD



SE RTU Solution 2024

Level-1 DFD



Explanation

- Student enters personal and academic details
- System verifies details using database
- Fee payment is processed and stored
- Registration slip/confirmation is generated

DFD Symbols

Symbol	Meaning
Circle	Process
Arrow	Data Flow
Rectangle	External Entity
Open Rectangle	Data Store

Applications of DFD

- Business systems
- MIS systems
- Database-driven applications
- Information systems

SE RTU Solution 2024

Advantages of DFD

- Easy to understand
- User-friendly
- Clearly shows data movement
- Helps detect missing processes

Limitations of DFD

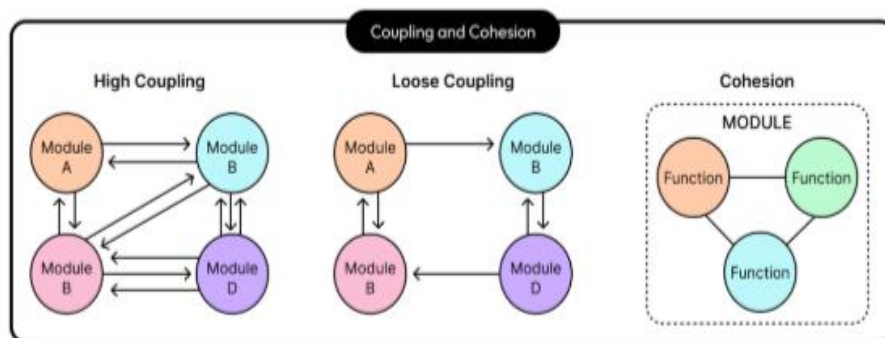
- Does not show control logic
- No timing information
- Complex systems require many levels

Q3. Explain Effective Modular Design in terms of Cohesion and Coupling with all its types and diagram.

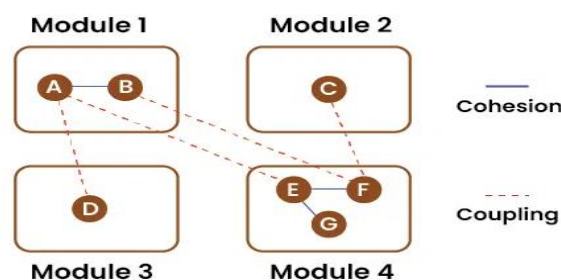
Effective Modular Design

Effective modular design is a software design approach in which a system is divided into **small, manageable, and independent modules**. Each module performs a specific function and interacts with other modules in a controlled manner.

The quality of modular design mainly depends on **Cohesion** and **Coupling**.



- **High Cohesion** (within a module)
- **Low Coupling** (between modules)



SE RTU Solution 2024

Cohesion

Cohesion is the measure of how **closely related the elements within a single module are**.

High cohesion is desirable, because it makes the module easy to understand, test, and maintain.

Types of Cohesion (Worst → Best)

1. Coincidental Cohesion (Worst) : Module performs unrelated tasks. No meaningful relationship among functions

Ex. Module A

- ├─ Print
- ├─ Sort
- └─ Calculate

2. Logical Cohesion : Related logically but executed based on a control flag

Ex. Module B

- ├─ Read File
- ├─ Write File
- └─ Open File

3. Temporal Cohesion : Tasks executed at the same time (e.g., initialization)

Ex. Exception handling with the help of catch function.

4. Procedural Cohesion : Tasks follow a specific sequence of execution

Step 1 → Step 2 → Step 3

Ex. Drive excess with permission.

5. Communicational Cohesion: Tasks operate on the same data

Module D

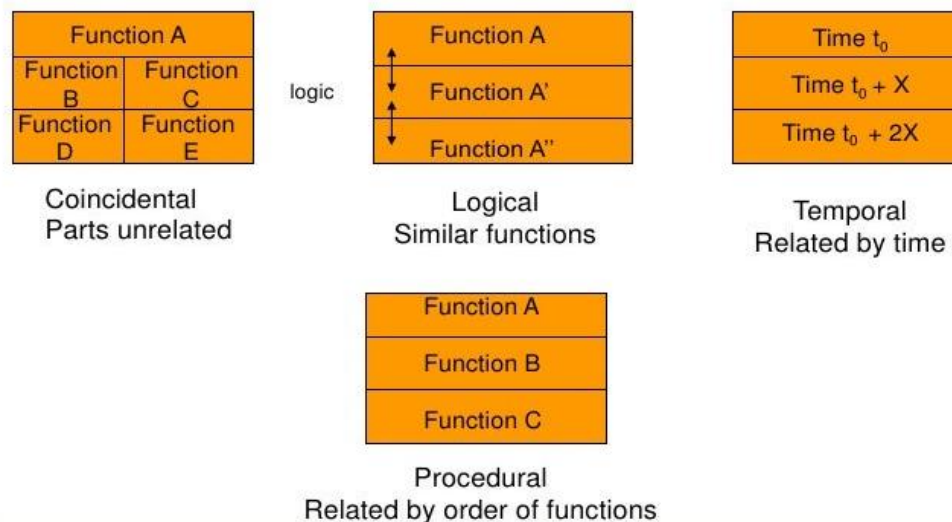
- ├─ Read Student Data
- └─ Update Student Data

6. Sequential Cohesion : Output of one task becomes input of another

SE RTU Solution 2024

Ex. Year-> Branch -> Subject-> Unit

7. Functional Cohesion (Best) : Module performs one single, well-defined function



Coupling

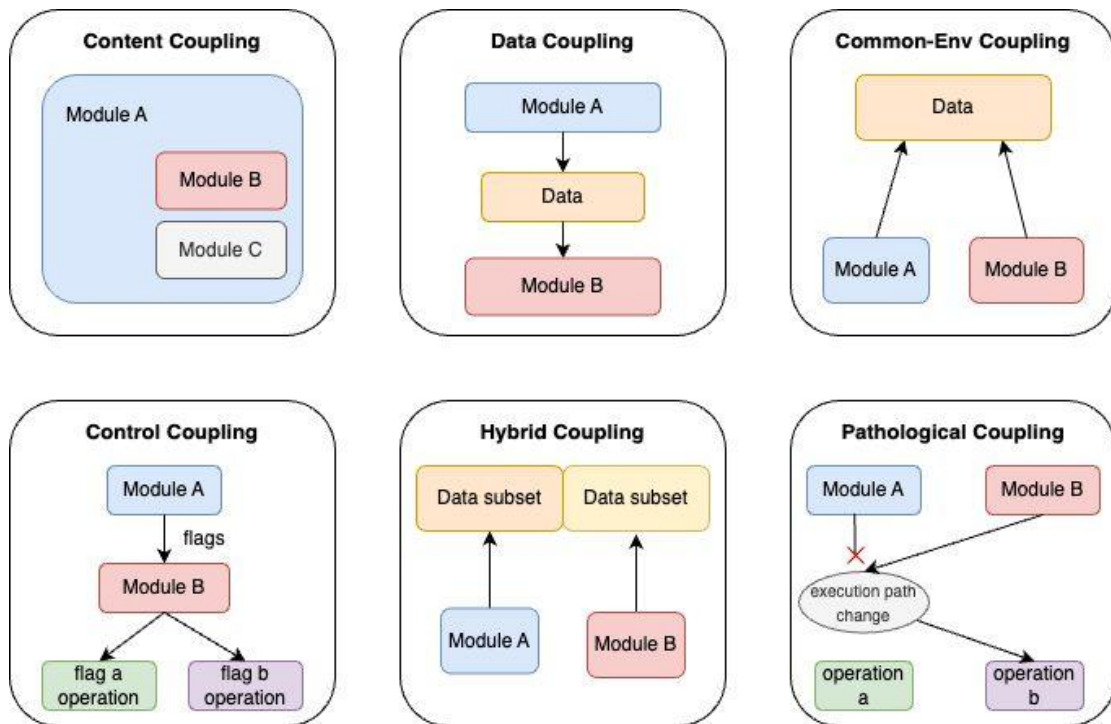
Coupling is the measure of **dependency between modules**.

Low coupling is desirable, as it reduces the impact of changes.

Types of Coupling (Worst → Best)

- 1. Content Coupling (Worst) :** One module directly accesses another module's internal data
- 2. Common Coupling :** Modules share global data
- 3. External Coupling :** dependency on hardware
- 4. Control Coupling :** One module controls the behavior of another using flags. Ex. Data sharing within two module
- 5. Stamp Coupling :** Modules share a composite data structure.Ex. Call by reference or address.
- 6. Data Coupling (Best) :** Modules communicate using simple data parameters. Ex. Call by value or data

SE RTU Solution 2024



Design Measurement

Design quality is measured using:

- Cohesion metrics
- Coupling metrics
- Complexity metrics
- Maintainability index

Q4. Define UML. How is it useful in Object-Oriented Modeling?
Explain the following:

(a) Use Case Diagram (b) State Chart Diagram

ANS: Unified Modeling Language (UML) is a **standard visual modeling language** used to **specify, visualize, construct, and document** the artifacts of a software system. It is mainly used in **object-oriented analysis and design (OOAD)** to represent the structure and behavior of a system in a clear graphical form. UML is independent of programming languages and development processes.

Usefulness of UML in Object-Oriented Modeling:

- Helps in **visualizing system architecture** before coding
- Represents **classes, objects, relationships, and interactions** clearly
- Improves **communication** among developers, designers, and clients

SE RTU Solution 2024

- Supports **abstraction and encapsulation** concepts of OOP
- Assists in identifying **classes, attributes, and methods**
- Facilitates **design documentation** for future maintenance
- Reduces development errors by early design validation

In object-oriented modeling, UML acts as a **blueprint** that transforms user requirements into a structured design, making software systems **easier to understand, develop, and maintain**.

1. Use Case Diagram

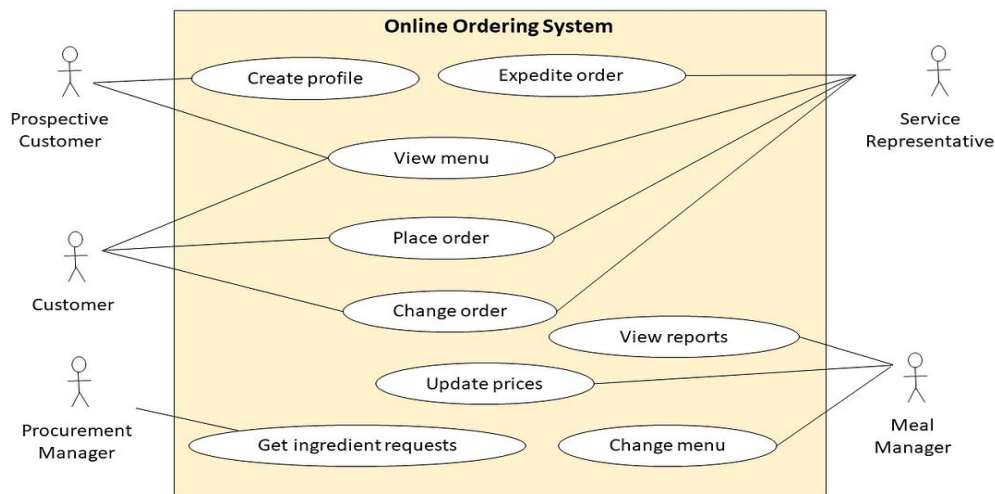
A **Use Case Diagram** is a **UML behavioral diagram** that represents the **functional requirements of a system** by showing **interactions between users (actors) and the system**.

It answers the question:

“What the system should do from the user’s perspective?”

Purpose of Use Case Diagram

- Captures **user requirements**
- Defines **system functionality**
- Provides a **high-level system view**
- Acts as a communication tool between users and developers



Main Components

1. Actor

- Represents a user or external system
- Can be a person, device, or another system

SE RTU Solution 2024

2. Use Case

- Represents a specific function or service
- Shown as an oval

3. System Boundary

- Defines the scope of the system
- Shown as a rectangle

4. Relationships

- Association
- Include
- Extend
- Generalization

Advantages

- Easy to understand
- User-centric
- Helps in requirement validation
- Useful in test case generation

Limitations

- Does not show internal logic
- No sequence or timing details

2. State Chart Diagram

A **State Chart Diagram** is a **UML behavioral diagram** that represents the **dynamic behavior of an object** by showing **different states and transitions** in response to events.

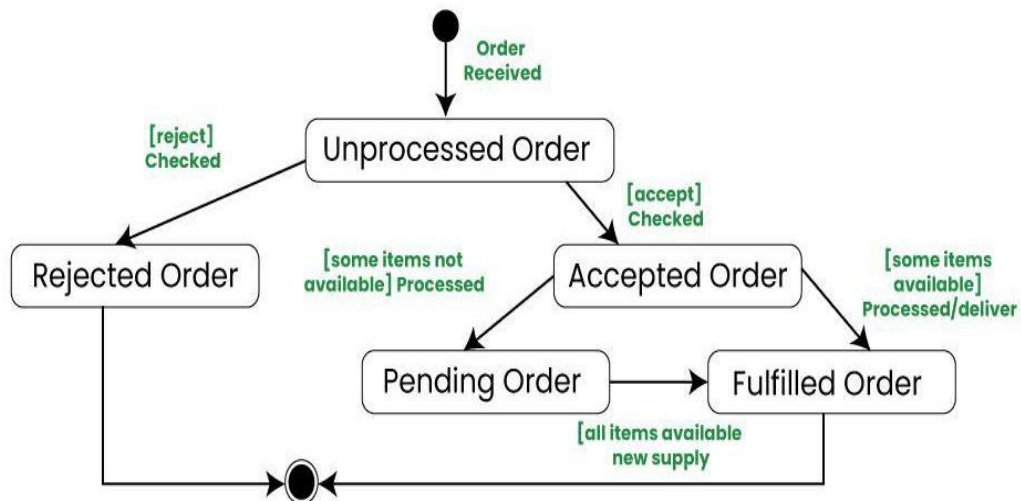
It answers the question:

“How does the object behave over time?”

Purpose of State Chart Diagram

- Models **object life cycle**
- Shows state changes due to events
- Useful for **real-time and embedded systems**

SE RTU Solution 2024



Main Components

1. State

- Condition in which an object exists
- Represented by a rounded rectangle

2. Event

- Trigger that causes state change

3. Transition

- Movement from one state to another
- Represented by arrows

4. Initial and Final State

- Initial state → solid circle
- Final state → double circle

Advantages

- Clearly shows object behavior
- Helps in system validation
- Useful for control systems

Limitations

- Complex for large systems
- Not suitable for data flow modeling

SE RTU Solution 2024

Q5. Compute the function point productivity, documentation, and cost per function for the following data:

Measurement Parameter	Count	Weighting Factor
No. of External Inputs (EI)	24	4
No. of External Outputs (EO)	46	4
No. of External Inquiries (EQ)	8	6
No. of Internal Logical Files (ILF)	4	10
No. of External Interface Files (EIF)	2	5

Given:

Effort = **36.9 Person-Months**

Technical documents = **265 pages**

User documents = **122 pages**

Cost = **\$7744 per month**

Processing factors = **4, 1, 0, 3, 3, 5, 4, 4, 3, 3, 2, 2, 4, 5**

Soln.

Calculate Unadjusted Function Points (UFP)

UFP= Sum(count× weight factor)

UFP=(24×4)+(46×4)+(8×6)+(4×10)+(2×5)= 378

Calculate Total Degree of Influence (TDI)

TDI=4+1+0+3+3+5+4+4+3+3+2+2+4+5=43

Calculate Value Adjustment Factor (VAF)

VAF=0.65+(0.01×TDI)

VAF=0.65+0.43=1.08

Calculate Adjusted Function Points (FP)

FP=UFP×VAF=378×1.08=408

SE RTU Solution 2024

Function Point Productivity

$$\begin{aligned}\text{Productivity} &= \text{FP}/\text{Effort} = 408/36.9 \\ &= 11.05 \text{ FP/Person-Month}\end{aligned}$$

Documentation per Function Point

Total documentation pages = technical pages + user document

$$\text{Total documentation pages} = 265 + 122 = 387$$

$$\text{Documentation/FP} = 387/408 = 0.95 \text{ pages/FP}$$

Cost per Function Point

Total cost = Effort $\times$ cost per month

$$\text{Total cost} = 36.9 \times 7744 = 285754$$

$$\text{Cost per function} = (\text{Total Cos})t/\text{FP} = 285754/408 = \$700.37 \text{ per FP}$$